Frontend Development Standards

Team Engineering Standard | Topic 2

Version 1.0 | Status: Proposed baseline | Applies to: React frontend teams

Purpose

A practical team reference for naming conventions, coding practices, Git branching, commit messages, Pull Requests, code reviews, and automated quality gates.

1. Purpose and scope

This document defines how frontend developers name code, write maintainable implementations, collaborate through Git, submit Pull Requests (PRs), and review changes. These standards are intended to make delivery predictable and code easier to maintain across the team.

- Apply these standards to new code and changed code. Avoid large unrelated refactors unless agreed.
- Follow existing repository and organization policies where they differ from this proposed baseline.
- The Frontend Lead owns the standard; the team can propose changes through a PR to this document.

Note: *This is a proposed baseline. Confirm branch names, required approvals, CI checks, and release steps against the team's actual GitHub and DevOps setup.*

2. Naming conventions

Use consistent, descriptive names so code can be understood without reading every implementation detail.

Item	Convention	Example
React component / component file	PascalCase	ProductCard.jsx / ProductCard
Custom hook	camelCase with use prefix	useProducts.js / useProducts()
Function	camelCase, verb-oriented	getProducts(), handleSubmit()
Variable	camelCase, descriptive	filteredProducts
Boolean	Question/state wording	isLoading, hasError, canSubmit
Constant	UPPER_SNAKE_CASE for true constants	API_TIMEOUT
Test file	Match source name	ProductCard.test.jsx
CSS Module	Match component name	ProductCard.module.css
Branch	lowercase, ticket-based	feature/UI-245-product-filters
Commit	Conventional, concise message	feat(products): add filters

- Prefer meaningful names over generic names such as data2, temp, or doStuff.
- Use handleX for event handlers (handleAddToCart) and onX for callback props (onAddToCart).
- Follow the repository's established file naming style; don't rename files solely for personal preference.
- Use UPPER_SNAKE_CASE for values that are truly constants, not every variable that happens not to change in one function.

3. Coding practices

Write code that is correct, readable, testable, and consistent with the agreed architecture. Optimize for clarity first; avoid clever or overly generic abstractions.

React and component practices

- Keep each component focused on a clear responsibility; extract a component when it improves clarity, reuse, testing, or independent work.
- Keep state as close as practical to where it is used. Lift state only when components need to share it.
- Use stable keys for rendered lists; do not use array indexes as keys when list items can be reordered or removed.
- Follow the Rules of Hooks. Do not call hooks conditionally or inside loops.
- Do not use effects to calculate values that can be derived during render. Use effects for synchronization with external systems.
- Use semantic HTML and accessible names for interactive controls.
- Handle loading, empty, success, and error states for asynchronous UI.

API and state practices

- Use the project's shared API client and feature service layer; avoid duplicating base URLs, headers, or request logic inside components.
- Use the agreed server-state tool (for example, TanStack Query) for API data and caching.

- Use Context or Redux Toolkit for shared client state only when justified by the app's needs.
- Do not keep multiple competing copies of the same source of truth without a deliberate synchronization strategy.
- Never commit secrets. Vite variables prefixed with VITE_ are exposed to client-side code and must not contain secrets.
- Handle HTTP errors consistently and avoid displaying stack traces or sensitive backend details to users.

Code quality and testing

- Use ESLint for code-quality rules and Prettier for consistent formatting.
- Add or update tests for important behaviour and bug fixes using the project's chosen test stack.
- Keep functions and components understandable; avoid premature abstraction and unnecessary state.
- Remove debug logs, dead code, unused imports, and commented-out blocks before merging.
- Run the relevant lint, tests, type checks (if TypeScript is adopted), and production build before opening a PR.

4. Git branching strategy

A branch isolates a line of work. The branch model below is a common Git Flow-style option; use the model agreed by your project and release process.

Branch	Purpose	Typical base / owner
main	Production-ready code	Initial repository branch; protected
develop	Integrates approved development work	Usually created from main during setup
feature/<ticket>-<short-name>	One feature or task	Created from develop by the assigned developer
release/<version>	Stabilize a planned release	Created from develop by release owner
hotfix/<short-name>	Urgent production fix	Created from main by authorized incident owner

Typical workflow

```
git switch develop
git pull origin develop
git switch -c feature/UI-245-product-filters

# Implement and test the task
git add .
git commit -m "feat(products): add product filters"
git push -u origin feature/UI-245-product-filters
```

- Create a feature branch when starting an assigned task; keep it focused on one logical change.
- Open a PR from the feature branch to the agreed integration branch, commonly develop in this model.
- Release branches are optional and should be created only when the release process requires stabilization.
- Create production hotfixes from the production code (commonly main), follow the emergency review/release process, and synchronize the fix back into active development/release branches.
- Delete merged feature branches when no longer needed, manually or through repository automation.
- Do not confuse Git branches with DEV/SIT/UAT/PROD environments. CI/CD determines which commit or artifact is deployed where.

Note: *Some teams use trunk-based development and do not keep a long-lived develop or release branch. Do not introduce this branch model without confirming the repository's current process.*

5. Commit conventions

Use short, action-oriented commit messages. A Conventional Commits-style pattern is a useful default.

```
<type>(<scope>): <short description>

Examples:
feat(products): add category filters
fix(auth): handle expired session
refactor(cart): simplify total calculation
test(products): cover empty results
chore(deps): update dependencies
```

- Use feat for a new feature, fix for a bug fix, refactor for a behaviour-preserving restructure, test for tests, and chore for maintenance.
- Keep commits logically focused and avoid committing generated output, secrets, or unrelated formatting changes.
- Follow any commit-signing or message rules mandated by the organization.

6. Pull Request (PR) standards

A PR asks the team to review and integrate changes from one branch into another. It is a quality checkpoint, not merely a merge button.

Every PR should include

- A clear title and linked ticket/task.
- A concise summary of what changed and why.
- Testing performed and relevant evidence (screenshots for visible UI changes where useful).
- Known limitations, migration notes, or configuration changes when relevant.
- A focused scope; separate unrelated features/refactors when practical.

Example PR description

```
Title: feat(products): add category filters

Ticket: UI-245

Summary:
- Add category dropdown to Products page.
- Update product query when category changes.
- Add loading, error, and empty states.

Validation:
- [ ] Category selection filters products.
- [ ] All Categories resets the filter.
- [ ] API failure displays a helpful message.
- [ ] Relevant tests, lint, and build pass.
```

Merge requirements

- Required CI checks must pass.
- The required number of qualified reviewers must approve.
- Blocking review conversations must be resolved according to repository policy.
- Changes must target the correct branch and satisfy any release requirements.
- Use the repository's agreed merge strategy (merge commit, squash, or rebase). Squash merge is often a reasonable default for focused feature PRs, but consistency matters more than the specific choice.
- Protect important branches against unauthorized direct pushes; configure GitHub rulesets/branch protection to enforce the policy.

7. Code review standards

Review for correctness, maintainability, security, accessibility, testing, and consistency with the agreed architecture. Review the change, not the developer.

Review checklist

- Requirements: Does the change meet the ticket and handle relevant edge cases?
- Architecture: Are responsibilities clear? Does the code use the agreed API and state patterns?
- React: Are hooks used correctly? Are list keys stable? Is state derived or duplicated appropriately?
- Quality: Are names clear? Is duplication avoided? Is dead code removed?
- Security: Are secrets absent? Is user input and authentication handled appropriately?
- Accessibility: Are semantic elements, labels, keyboard behaviour, and focus considered?
- Tests: Are important behaviours covered? Do required checks and the build pass?

How to give feedback

- Be specific and explain why a change is needed.

- Separate blocking issues from optional suggestions.
- Ask questions when intent is unclear instead of assuming the author made a mistake.
- Suggest a direction or example where possible.
- Re-review changes after requested fixes and confirm blocking discussions are resolved.

```
Example:
"Could we use the existing useProducts hook here? This keeps the page consistent with our TanStack Query pattern an
d reuses caching and error handling. Please also confirm that the query updates when the selected category changes.
"
```

Who reviews and merges?

- The author opens the PR and responds to feedback.
- A qualified peer or feature owner reviews it.
- The Frontend Lead focuses on architecture, high-risk changes, shared components, and cross-feature impact.
- Only authorized contributors merge, after the configured checks and approvals pass.

8. Automation and enforcement

Documented standards work best when repeatable checks are automated.

Check	Typical tool / mechanism
Linting	ESLint; npm run lint
Formatting	Prettier; npm run format:check
Tests	Vitest/Jest + React Testing Library
Type checking	TypeScript compiler if adopted
Production build	npm run build
PR checks	GitHub Actions or the organization's CI
Approval and branch restrictions	GitHub rulesets / branch protection
PR consistency	Repository PR template

- Store configuration and lockfiles in the repository so the team shares the same setup.
- Run checks locally before opening a PR and repeat them in CI.
- Configure required status checks in the repository settings; merely adding a workflow does not automatically block merging.
- Do not require tools or scripts that are not configured in the actual repository.

9. Definition of done

- The implementation matches the agreed requirements and architecture.
- Names, formatting, and code structure follow team conventions.
- Relevant tests are added or updated and pass.
- Lint, type checking (if adopted), and production build pass.
- Loading, empty, and error states are addressed where applicable.
- PR description, ticket link, and relevant UI evidence are included.
- Required reviews are approved and blocking discussions are resolved.
- Changes are merged using the agreed branch and release process.

10. Ownership and updates

The Frontend Lead owns this baseline and facilitates agreement on changes. Developers are responsible for following it, testing their changes, and raising questions early. Reviewers help enforce the standard constructively.

Suggested repository path: docs/frontend-development-standards.md. Update this document through a PR when team decisions change.